

Self-Hosted Multi-Tier Cloud for SQL Server-Backed Application

Multi-Tier Cloud for SQL-Backed Application: Instructions

1 - Download ProxMox ISO, I chose to use version 8.4

<https://www.proxmox.com/en/downloads>

2 - Burn ISO onto a USB drive using a program like balenaEtcher or Rufus. Restart target pc and hold down the boot option for you specific BIOS. Choose the ProxMox installer and continue.

3 - The networking page should automatically find your local subnet and assign IP's accordingly.

If it is empty you will have to do this manually. On home PC, Google WhatsMyIP to find your local network, like 192.168.0.0/24. Fill in this blanks accordingly, it should look something like this:

```
Hostname: cloud.local
IP Address: 192.168.0.50/24
Gateway: 192.168.0.1
DNS Server: 192.168.0.1
```

4 - (optional) You're home router will likely change this IP dynamically, in most cases you do not want this. Go into your router's setting and set up a DHCP reservation for the IP address you just entered. This will prevent it from changing and can save you headaches in the future.

5 - Once installation is complete you will be greeted by a login terminal along with a link and a port number that you can paste into your browser to access the ProxMox dashboard. Enter the link into browser on your main PC. Log in.

Note: the default username is root.

6 - ProxMox has shared a post installation script to make setting up repositories easier. Find the post install script and run it. Keep defaults and select the free version when asked.

<https://community-scripts.org/scripts?id=post-pve-install>

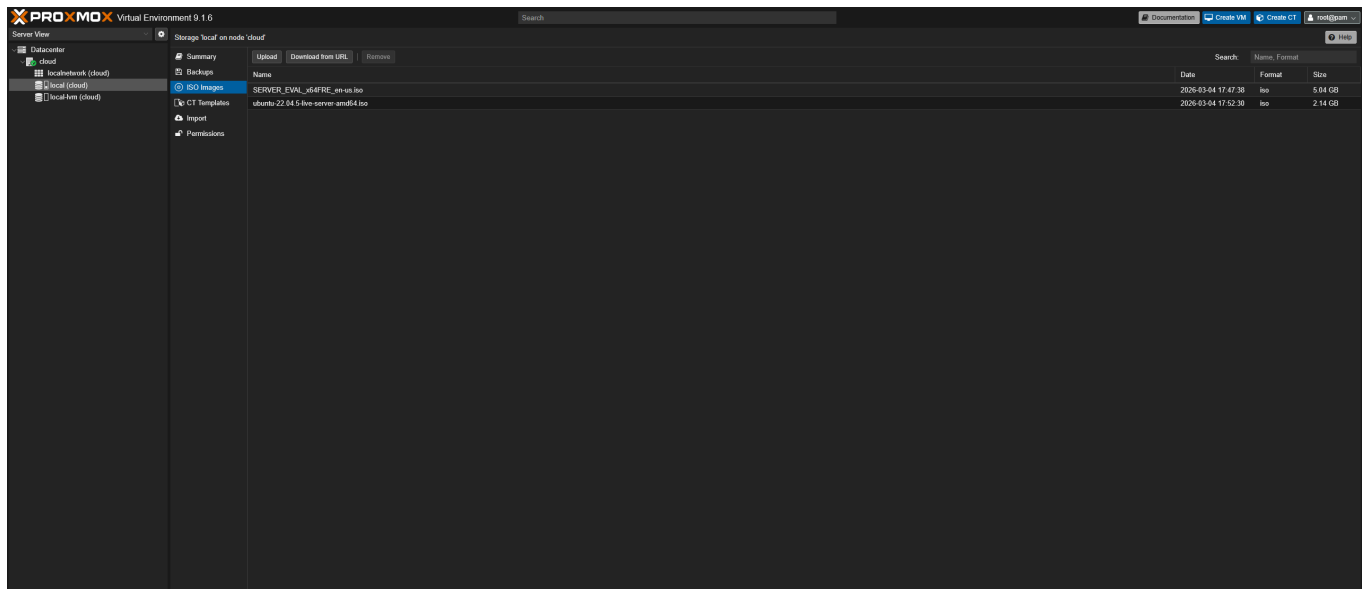
7 - For this project we will need a Ubuntu Server VM, and a Window Server VM. Download the ISO's for each. Save them in a place you can easily get to.

Windows Server 2022 - <https://www.microsoft.com/en-us/evalcenter/download-windows-server-2022>

Ubuntu Server 22.04 - <https://releases.ubuntu.com/jammy/>

8 - Back on ProxMox navigate to Datacenter -> cloud -> local(cloud) -> ISO Images, click upload and select the ISO's.

ISO images uploaded



9 - Before we create a VM we need to setup a private virtual bridge that the web-server VM and Database VM will use to communicate with each other. Navigate to cloud -> System -> Network, click the create dropdown and select Linux Bridge. Name the bridge vmb1, and click create.

10 - We will now configure the Ubuntu VM. Click on Cloud then select Create VM on the top right. Enter the following and keep the defaults otherwise:

General:

Name: web-server

Continue

OS:

Ubuntu 22.04 Server

Continue

System:

Enable: QEMU (for cleaner shutdowns)

Continue

Disks:

Bus/Device: VirtIO BLOCK

Storage: local-lvm

Disk Size: 20GB

Continue

CPU:

Cores: 2

Type: host

Continue

Memory:

Memory: 4096 MB

Continue

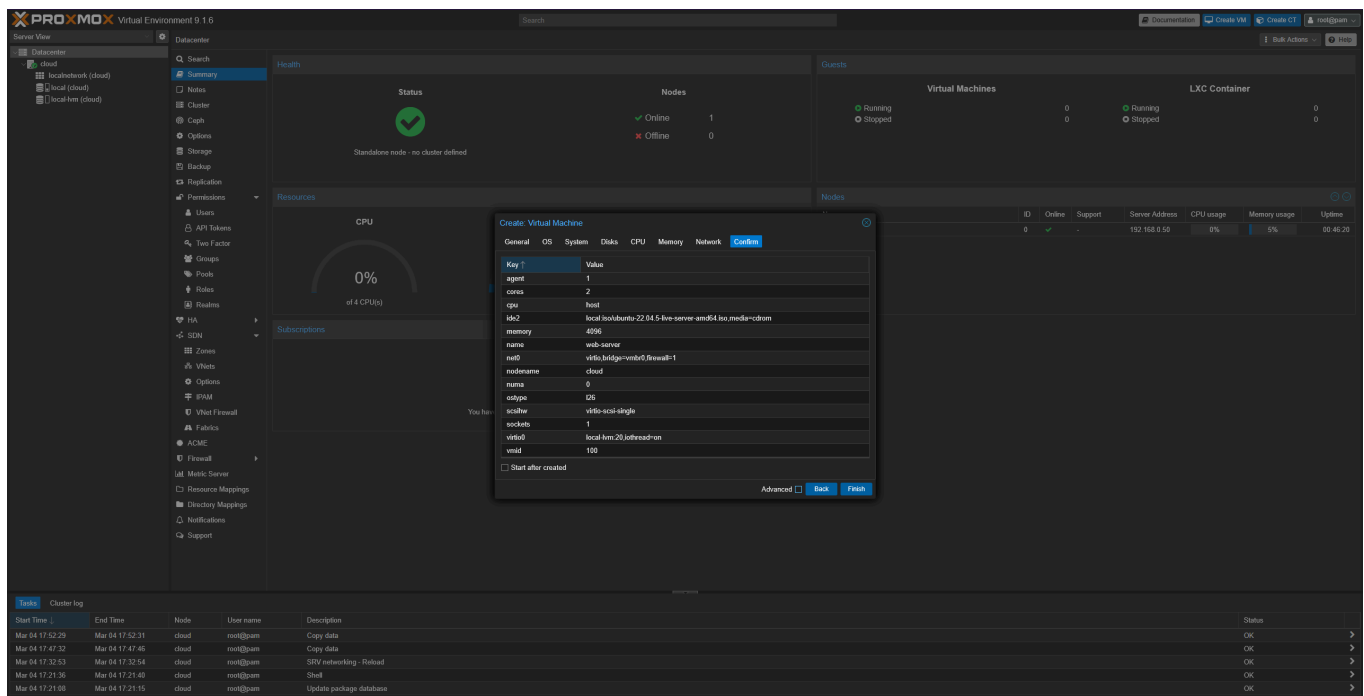
Network:

Bridge: vmbri0 (public)

Model: VirtIO

Continue

Review configurations and click finish.



Note: Using 'host' exposes CPU features to the guest OS. Enabling this should improve virtualization performance.

Note: VirtIO is a paravirtualized device driver standard. This allows more efficient communication between host and guest OS. The guest OS is now aware that it is being virtualized and will communicate with the hypervisor accordingly.

Note: QEMU is used to exchange information between the host and the guest. It is essential for proper shutdowns and restarts.

13 - Before we start the VM we need to add the vmbri1 bridge to the VM.

Navigate to Datacenter -> cloud -> Hardware, click the Add dropdown and select Network Device.

```
Bridge: vmbr1
Select: Add
```

12 - We are now ready to start the VM.

Navigate to Datacenter -> cloud -> web-server, then click start on the type right. Click on Console to go through the Ubuntu installer. We will have to manually configure the vmbr1 network.

Network Configuration:

```
ens18 (vmbr0): keep default
ens19(vmbr1):
    Subnet: 10.10.10.0/24
    Address: 10.10.10.10
Continue
```

Storage Configuration:

```
Select: Use entire disk
Continue
```

SSH Configuration:

```
Select: Install Open SSH server
Continue
```

Manual network configuration

The screenshot shows the Proxmox VE 9.1.6 interface. The main window displays the 'Network configuration' for VM 100 (web-server). The configuration is as follows:

```
NAME    TYPE    NOTES
[ ens18  eth    -
  DHCPv4  192.168.0.24/24
          bc:24:11:56:21:76 / Red Hat, Inc. / Virtio network device

[ ens19  eth    -
  disabled autoconfiguration failed
          bc:24:11:5c:5b:35 / R

[ Create bond ]
```

The 'Edit ens19 IPv4 configuration' dialog is open, showing the following settings:

- IPv4 Method: Manual
- Subnet: 10.10.10.0/24
- Address: 10.10.10.10
- Gateway: (empty)
- Name servers: (empty)
- Search domains: (empty)

The bottom of the interface shows a 'Tasks' table with the following data:

Start Time	End Time	Node	User name	Description	Status
Mar 04 18:32:05		cloud	root@pam	VMCT 100 - Console	
Mar 04 18:32:04	Mar 04 18:32:15	cloud	root@pam	VMCT 100 - Console	Error: connection timed out
Mar 04 18:32:01	Mar 04 18:32:05	cloud	root@pam	VM 100 - Start	OK
Mar 04 18:07:24	Mar 04 18:07:26	cloud	root@pam	VM 100 - Create	OK
Mar 04 17:52:29	Mar 04 17:52:31	cloud	root@pam	Copy data	OK

Wait for install to complete.

You may be asked to remove the media. If so, navigate to Datacenter-> cloud -> web-server -> hardware. Click on CD/DVD drive and select remove.

Proceed.

13 - Lets now install Nginx to host our web server. In the terminal run:

```
sudo apt update && sudo apt upgrade -y
```

Then:

```
sudo apt install nginx -y
```

Check that Nginx is active:

```
sudo systemctl status nginx
```

Now, navigate to your browser to test connectivity.

```
Enter: http://{server IP address}/
```

You should see the default landing page.

14 - Next configure firewall rules using Ubuntu's UFW (Uncomplicated Firewall). We will be allowing SSH only from our local subnet, as well as allowing HTTP.

```
sudo ufw allow from {your subnet here} to any port 22
```

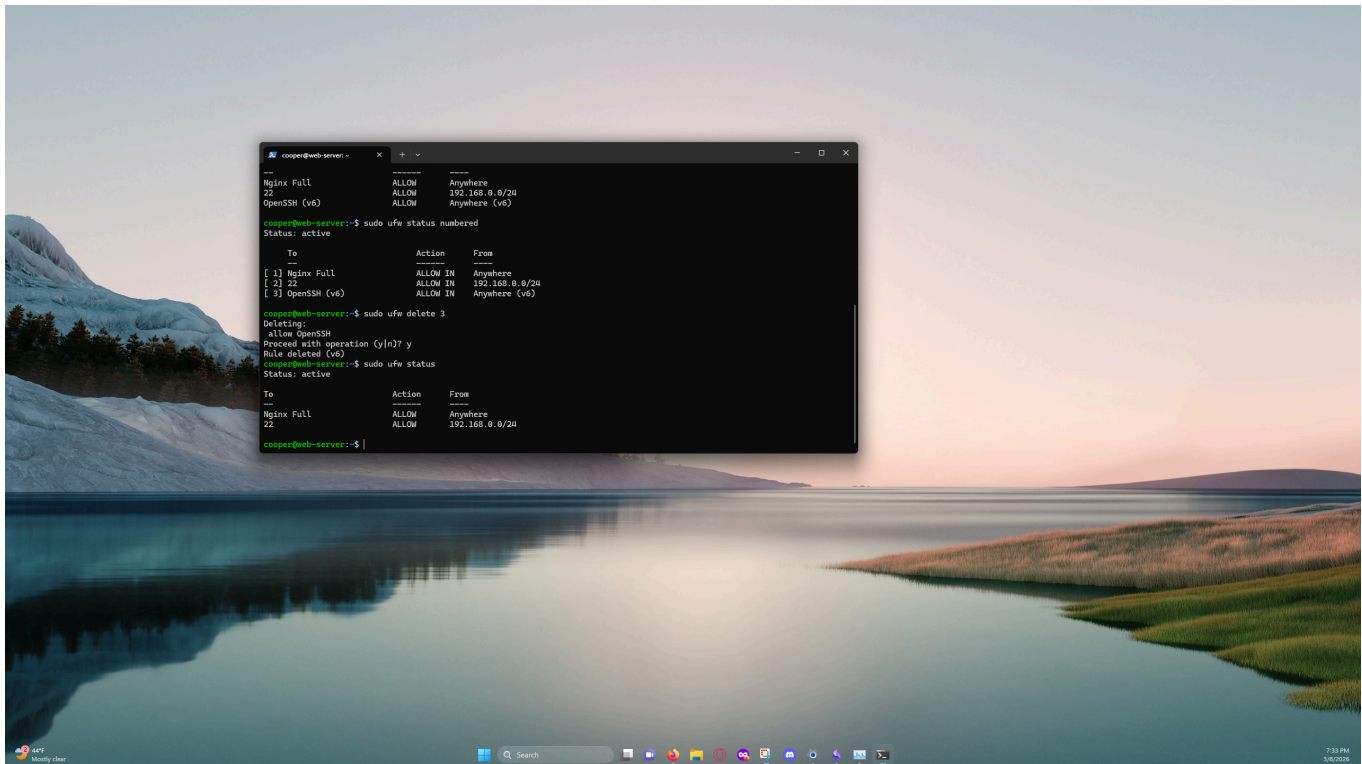
```
sudo ufw allow 'Nginx Full'
```

```
sudo ufw enable
```

```
sudo ufw status
```

You should see only two ALLOW rules. This means that the rest of traffic is denied by default with UFW.

Expected result of `sudo ufw status`



15 - Now lets setup the Windows VM, but first we need to grab the VirtIO driver ISO for Windows. Then upload the ISO, reference Step 8.

<https://fedorapeople.org/groups/virt/virtio-win/direct-downloads/archive-virtio/?C=M;O=D>

Click Create VM as before:

General:

Name: database-server

OS:

ISO Image: {Windows ISO}

Type: Microsoft Windows

Version: 11/2022/2025

Check: Add additional drive for VirtIO drivers

ISO Image: {VirtIO ISO}

Continue

System:

Check: Qemu Agent

Continue

Disk:

Bus/Device: VirtIO Block

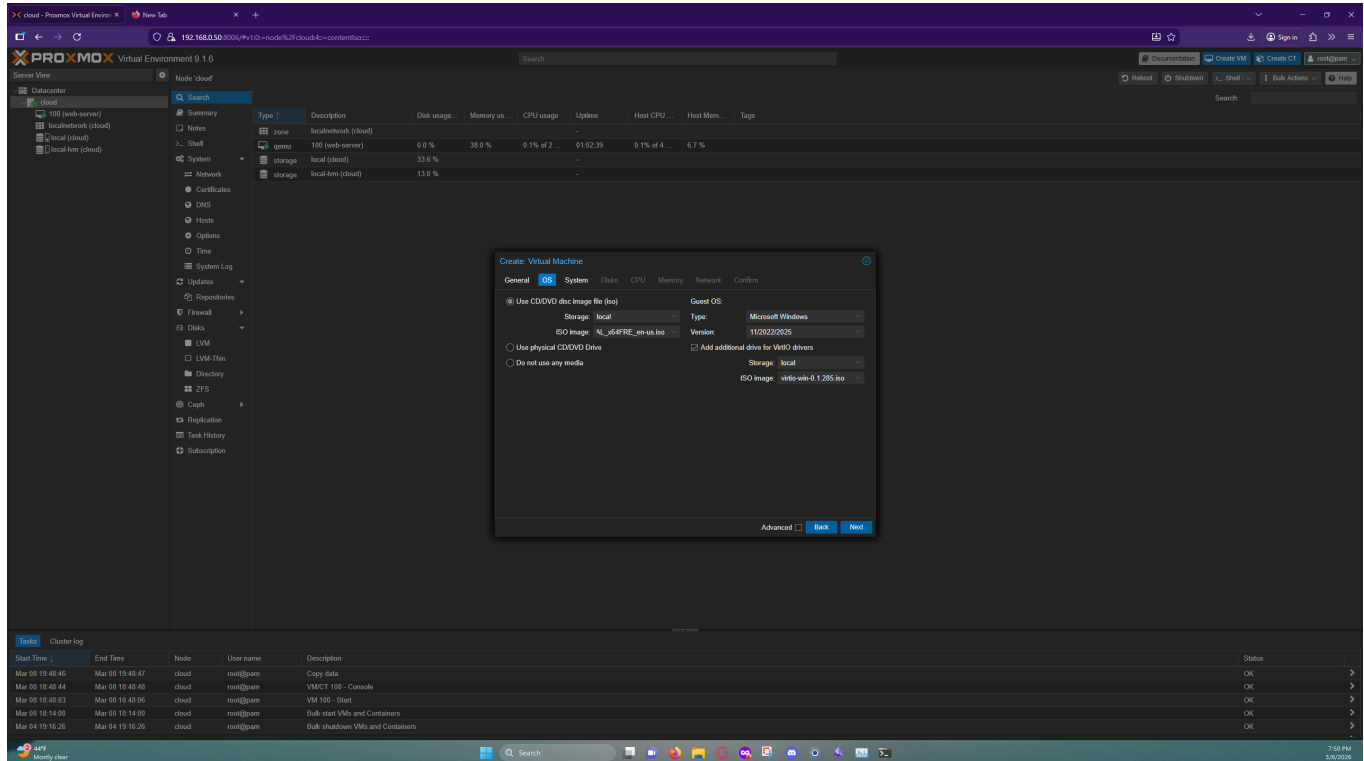
Disk Size: 50 GB

Continue

CPU:

```
Cores: 4
Type: host
Continue
Memory:
8182 MB
Continue
Network:
Bridge : vmbr1
Model: VirtIO
```

Adding ISO for Windows & VirtIO



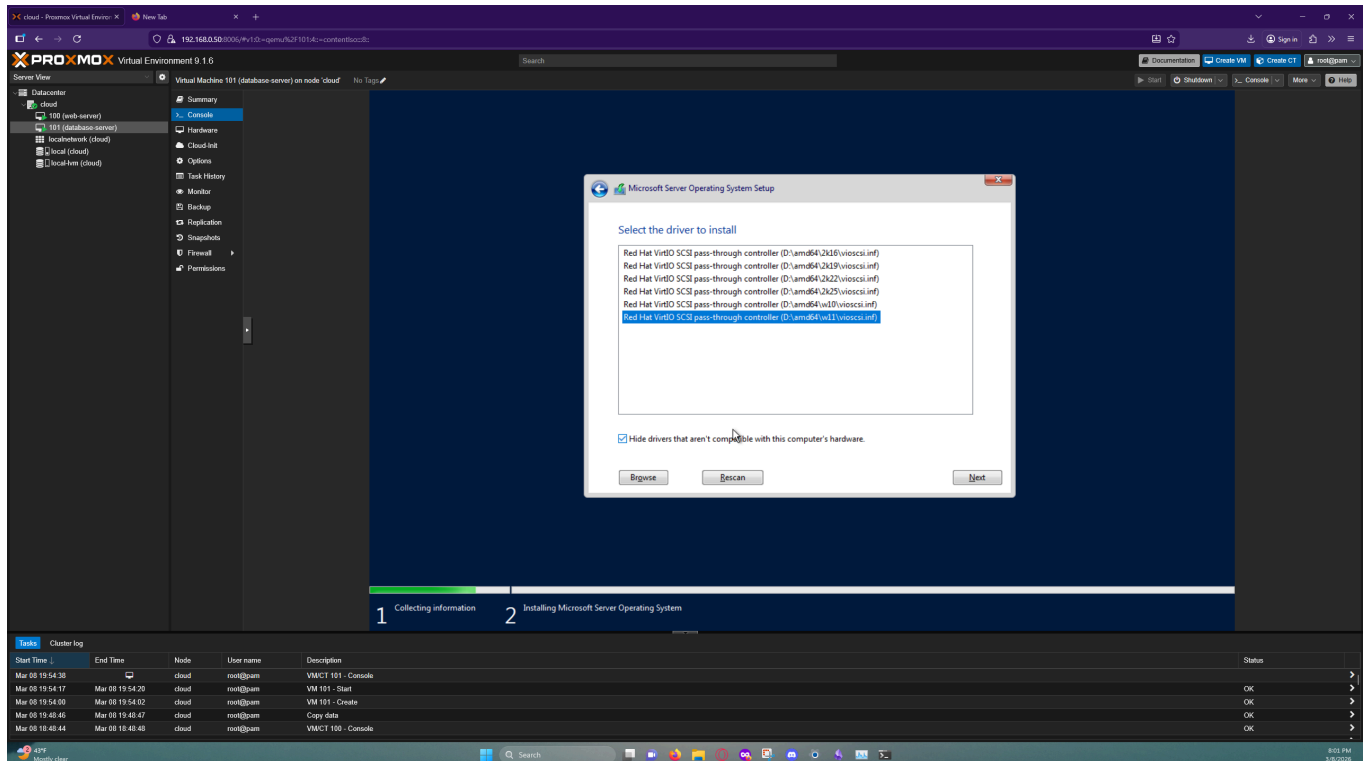
Review configurations and select finish.

You can now start the VM. Windows Installation will be simple. When you get the option to add driver:

```
Select: Red Hat VirtIO SCSI pass-through controller
```

After installation is complete you should be able to login into Windows.

Adding VirtIO SCSI pass-through controller



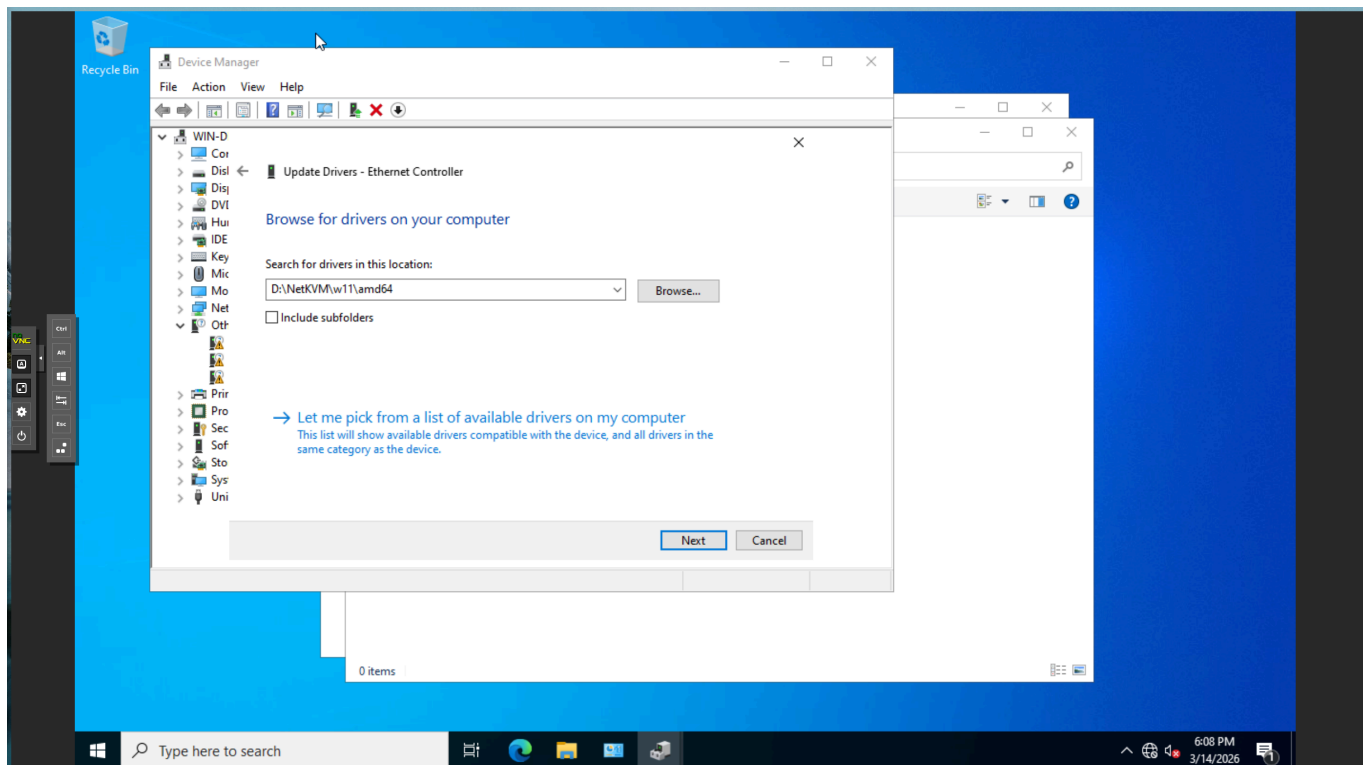
16 - First, we need to setup our VirtIO controller.

Go to: Device Manger -> Ethernet Controller -> Update Drivers

Click Browse, then select the CD Drive with the VirtIO ISO.

Navigate to \NetKVM\w11\amd64, click next.

This installs VirtIO Drivers.



Lets install the VirtIO Guest Tools to increase virtualization performance.

Navigate to File Explorer -> CD Drive with you VirtIO ISO

Run:

```
virtio-win-x.x.xxx.exe
```

Click through the installation and keep the defaults. You may need to restart after.

This installs several drivers to improve virtualization performance on Windows.

17 - Now that the drivers are installed we can setup a static IP for our database-server.

Go to: Control Panel -> Network & Internet -> Network & Sharing Center -> Change adapter settings.

You should see the Red Hat VirtIO Ethernet Adapter

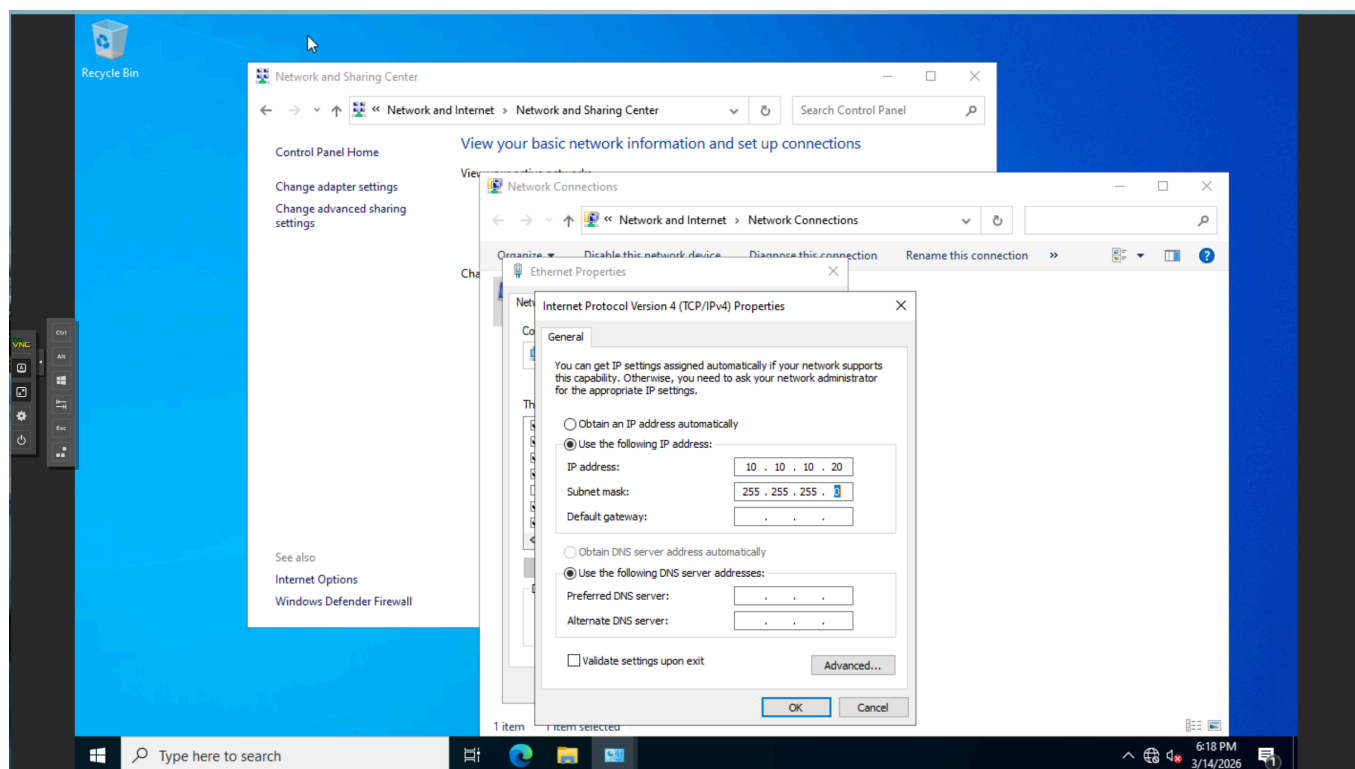
Right Click -> Properties

Internet Protocol Version 4 -> Properties

Enter:

```
IP Address: 10.10.10.20  
Subnet Mask: 255.255.255.0
```

Click OK



In Command Prompt, run:

```
ipconfig
```

You should see you configured IPv4 address.

18 - Test connectivity between web-server and database-server

In Windows, Command Prompt, run:

```
ping 10.10.10.10
```

You should see a response

In Ubuntu, run:

```
ping 10.10.10.20
```

If this does not work, but the connection worked the other way, this means we need to allow incoming pings on Windows

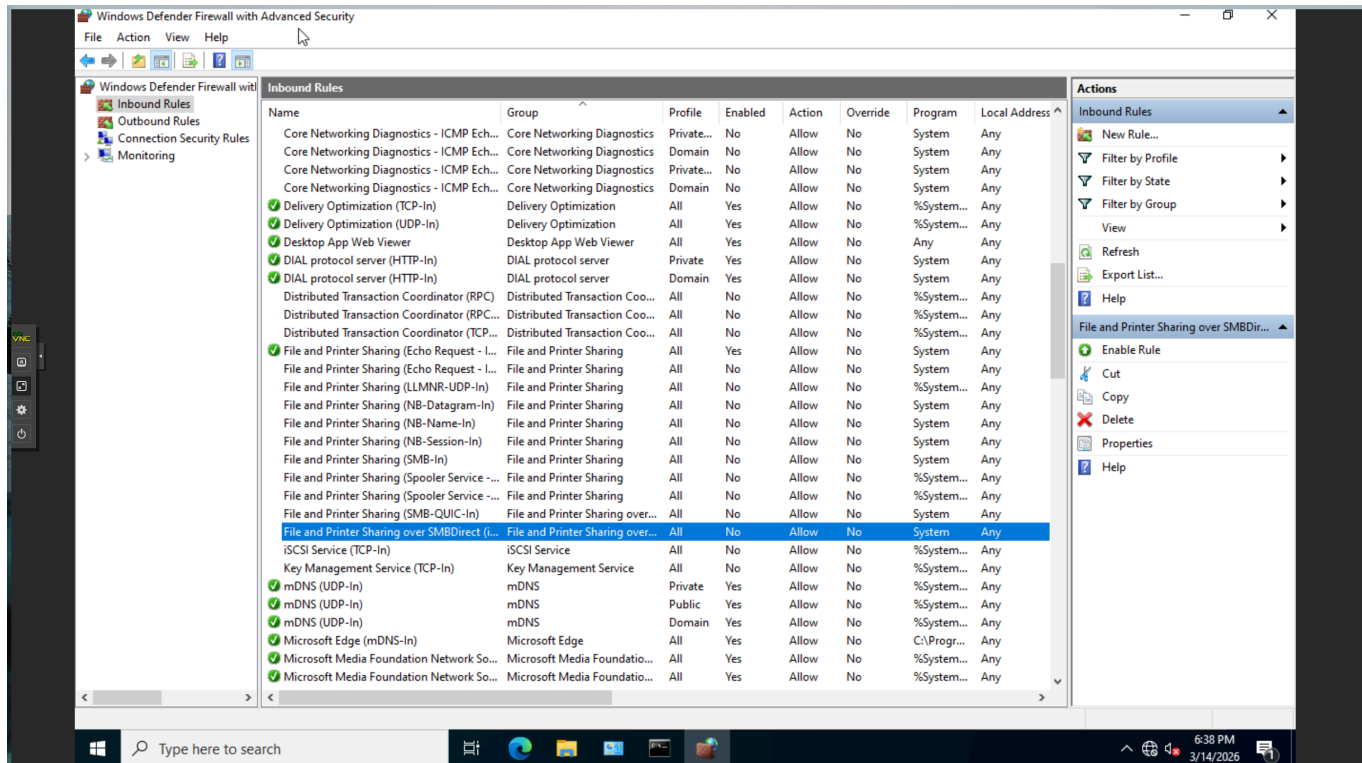
```
Click Start -> Windows Defender Firewall with Advanced Security -> Inbound
```

Rules

Find the rule called 'File and Printer Sharing (Echo Request - ICMPv4-In)'

Right Click -> Enable.

Try the ping again, this time it should work.



19 - It is time to download SQL Server Developer Edition. By design this PC cannot access the public network, so we need to add a temporary bridge to use the browser.

In ProxMox:

```
database-server -> Hardware  
Add -> Network Device  
Bridge: vmbr0
```

Open the browser and look up SQL Server Developer Edition, download the exe, and run the installer.

After install is complete, you should see a button that says 'Download SSMS,' click that to download SQL Server Management Studio.

Delete the temporary bridge.

20 - Let's make it so the web-server can access our SQL database. We will then configure a firewall rule so that **only** the web-server can access the SQL database.

Open SQL Server Configuration Manager

```
SQL Server Network Configuration -> Protocols for MSSQLSERVER
```

Then right click TCP/IP

```
Select: Enable
```

Create a firewall for inbound traffic over port 1433

Open Windows Defender -> Inbound

Click New Rule:

```
Select Port
    Click Next
Select TCP
Specific local ports: 1433
    Click Next
Allow the connection
    Click Next
Uncheck: public
    Click Next
Name: Allow SQL from Web Server
    Click Finish
```

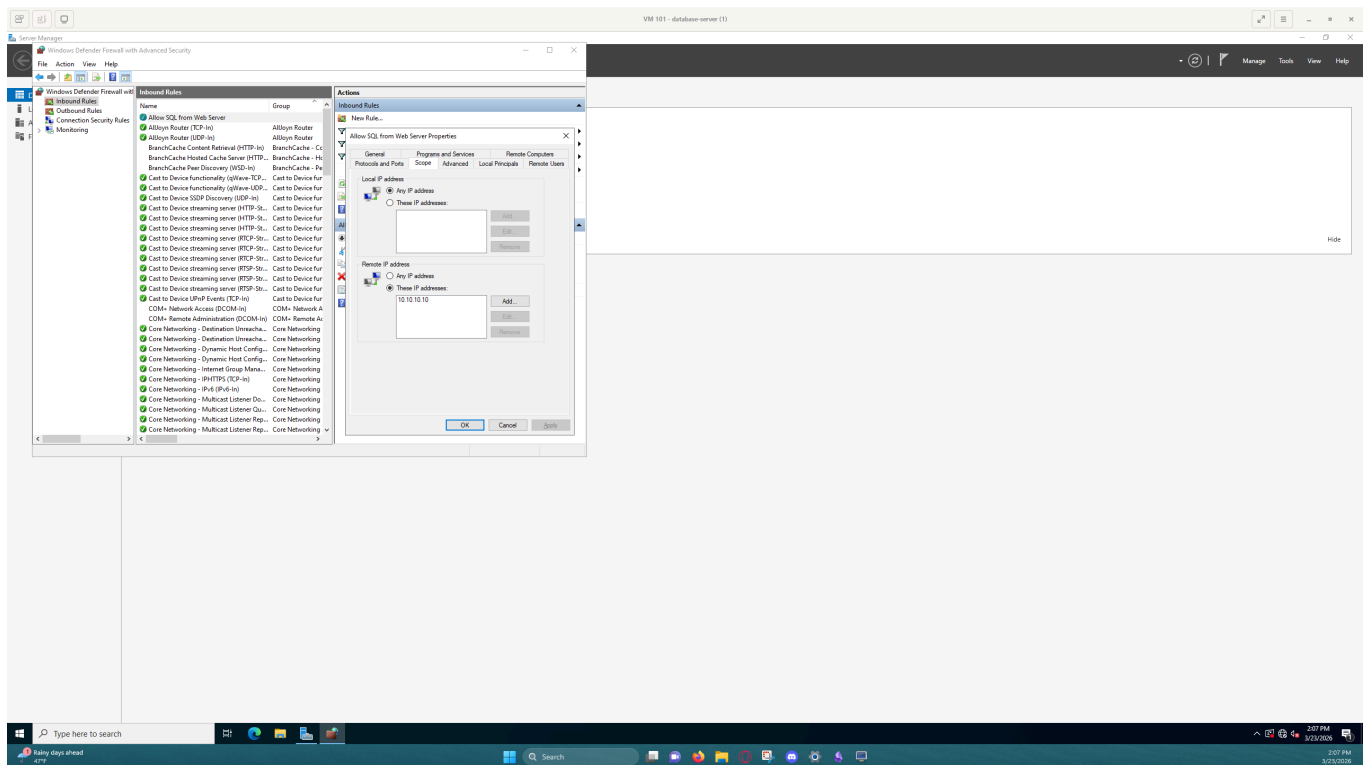
Now, we refine that rule to only allow traffic from the web-server

Right click the rule you just created and select Properties

Under Scope

```
Remote Addresses: 10.10.10.10
```

Click Finish.



Upon installation, it is possible that Windows set the network configuration to Public. This will cause problems on the next step.

To check, open PowerShell and run:

```
Get-NetConnectionProfile
```

If it says public, then run:

```
Set-NetConnectionProfile -NetworkCategory Private
```

21 - Test connectivity & firewall rules.

From web-server, run:

```
nc -zv 10.10.10.20 1433
```

You should get a response. This proves that web-server can access the SQL database

Now, from your main PC, assuming it's running Windows:

Open PowerShell, then run:

```
Test-NetConnection -ComputerName 10.10.10.20 -Port 1433
```

The connection will time-out. This proves that database-server is isolated.

22 - Now that the infrastructure is running and secure, you are able to create your database inside SQL Server Manager Studio, and import the data you need.

At this point, I'll show an example on how you can use this infrastructure to connect a Flask application to the SQL Server, creating a website where in which a user will be able to click a button, and return results from inside the SQL Server.

1 - The Flask app needs to connect with a SQL username/password in SSMS:

```
USE master
GO
CREATE LOGIN flaskuser WITH PASSWORD = "YourPassword"
```

Next map the user to the database:

```
USE *yourdatabase*;
GO

CREATE USER flaskuser FOR LOGIN flaskuser;
GO

ALTER ROLE db_datareader ADD MEMBER flaskuser;
GO
```

The db_databasereader role allows the user to run SELECT queries only.

Allow SQL authentication in SQL Server:

Right click your server in Object Explorer

```
Click properties -> security
```

```
Select : SQL Server and Windows Authentication mode
```

2 - Install Flask, Python packages, unixodbc and pyodbc on web-server.

On web-server, run:

```
sudo apt install -y python3 python3-pip python3-venv unixodbc unixodbc-dev
```

Then, install pyodbc:

```
curl https://packages.microsoft.com/keys/microsoft.asc | sudo gpg --dearmor -o /usr/share/keyrings/microsoft-prod.gpg

echo "deb [arch=amd64 signed-by=/usr/share/keyrings/microsoft-prod.gpg] https://packages.microsoft.com/ubuntu/22.04/prod jammy main" | sudo tee /etc/apt/sources.list.d/mssql-release.list

sudo apt update

sudo ACCEPT_EULA=Y apt install -y msodbcsql18
```

That concludes the installation requirements.

NOTES:

- python3 runs Flask
- pip installs Python packages
- venv isolates the project
- unixodbc is an ODBC manager that Linux uses to talk to SQL server
- pyodbc allows Python programs to interact with ODBC's

3 - Open port 5000 for the webpage.

Run:

```
sudo ufw status
sudo ufw allow 5000/tcp
sudo ufw reload
```

This is the port that Flask will use to host the webpage.

3 - Create the Flask application:

Create a project folder, run:

```
mkdir ~/databaseflask
cd ~/databaseflask
```

Create a virtual environment, run:

```
python3 -m venv venv
```

Then activate it:

```
source venv/bin/activate
```

Install Python packages into virtual environment:

```
pip install flask pyodbc
```

Now, let's create the Flask application. Under each of the Flask routes named *QueryX*, paste your desired SQL query in between the quotes.

Under DB_CONFIG, make sure to make necessary changes to *database* and *password*

```
nano app.py
```

Then paste:

```
from flask import Flask, render_template
import pyodbc
app = Flask(__name__)

DB_CONFIG = {
    "server": "10.10.10.20",
    "database": "YourDatabaseName!",
    "username": "flaskuser",
    "password": "YourStrongPassword123!",
    "driver": "ODBC Driver 18 for SQL Server"
}

def get_connection():
    conn_str = (
        f"DRIVER={{DB_CONFIG['driver']}};"
        f"SERVER={DB_CONFIG['server']};"
        f"DATABASE={DB_CONFIG['database']};"
        f"UID={DB_CONFIG['username']};"
        f"PWD={DB_CONFIG['password']};"
        "Encrypt=yes;"
        "TrustServerCertificate=yes;"
    )
    return pyodbc.connect(conn_str)

@app.route("/")
def home():
```



```

        return render_template("home.html")

@app.route("/query1")
def query1():
    query = """
    INSERT QUERY HERE
    """

    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(query)
        rows = cursor.fetchall()
        columns = [column[0] for column in cursor.description]
    return render_template("results.html", title="Query1 Title",
columns=columns, rows=rows)

@app.route("/query2")
def query2():
    query = """
    INSERT QUERY HERE
    """

    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(query)
        rows = cursor.fetchall()
        columns = [column[0] for column in cursor.description]
    return render_template("results.html", title="Query2 Title",
columns=columns, rows=rows)

@app.route("/query3")
def query3():
    query = """
    INSERT QUERY HERE
    """

    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(query)
        rows = cursor.fetchall()
        columns = [column[0] for column in cursor.description]
    return render_template("results.html", title="Query3 Title",
columns=columns, rows=rows)

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000, debug=True)

```

Save and Exit.

NOTES:

Each App Route maps the URLs to a specific function that will handle the logic for that URL.

Ex. Query1 maps to www.yourdatabase.com/query1

<https://www.geeksforgeeks.org/python/flask-app-routing/>

Optional: Feel free to change each Flask route's name, and the title located at the bottom of each Flask route. Just be sure to make it consistent throughout.

4 - Create the HTML for the landing page and the results page.

Create the templates folder:

```
mkdir templates
```

Create the landing page:

```
nano templates/home.html
```

Paste:

```
<!DOCTYPE html>
<html>
<head>
  <title>Database Demo</title>
</head>
<body>
  <h1>Database Demo</h1>
  <p>Select a query to run:</p>

  <ul>
    <li><a href="/query1">Query1 Title</a></li>
    <li><a href="/query2">Query2 Title</a></li>
    <li><a href="/query3">Query3 Title</a></li>
  </ul>
</body>
</html>
```

Now, create the results page:

```
nano templates/results.html
```

Paste:

```
<!DOCTYPE html>
<html>
<head>
  <title>{{ title }}</title>
  <style>
    table, th, td {
      border: 1px solid black;
      border-collapse: collapse;
      padding: 6px;
    }
  </style>
</head>
<body>
  <h1>{{ title }}</h1>

  <table>
    <tr>
      {% for col in columns %}
        <th>{{ col }}</th>
      {% endfor %}
    </tr>

    {% for row in rows %}
      <tr>
        {% for value in row %}
          <td>{{ value }}</td>
        {% endfor %}
      </tr>
    {% endfor %}
  </table>

  <br>
  <a href="/">Back to Home</a>
</body>
</html>
```

4 - Run the app and test the webpage.

While inside the project folder with the virtual environment still active

Run:

```
python app.py
```

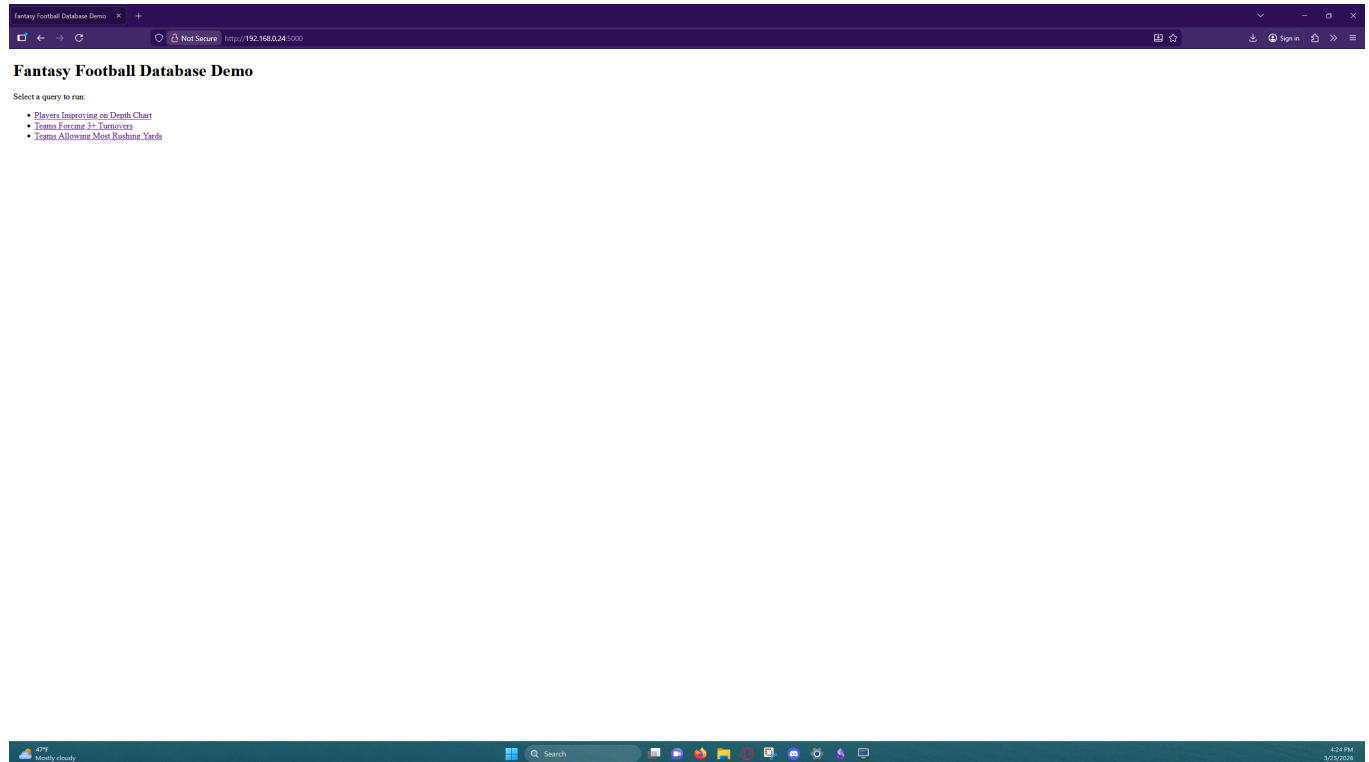
You should see a line that says:

```
Running on http://192.168.0.24:5000
```

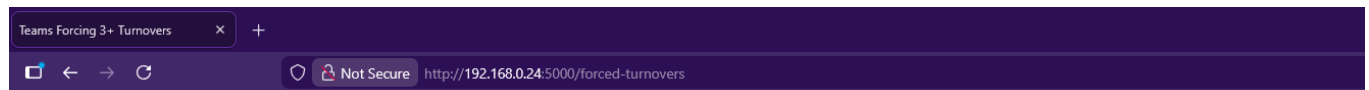
Copy that and paste it into your web browser.

You should now see your webpage, and when you select a query you should be taken to the results page, which is the data that Flask queried your SQL Server for.

Landing page



Results Page



Teams Forcing 3+ Turnovers

TEAM_NAME	TOTAL_TURNOVERS_FORCED
San Francisco 49ers	4

[Back to Home](#)

Troubleshooting:

- Double check to make sure your username, password and database name in app.py matches the SQL commands you ran to create them in the beginning of this section.
- Double check firewalls and ensure that you can still run `nc -zv 10.10.10.20 1433` from web-server
- Make sure that Windows did not switch back to public networking (which may happen after a restart)
- Lastly, make sure that your database is up and running inside SMSS